

LogMyLife V1

FINAL ANTIGRAVITY MASTER SPECIFICATION

Purpose: This document is the implementation contract for Antigravity. Follow the rules and acceptance criteria before adding new features or changing the core behavior.

Core philosophy: The user plans intentional activities. LogMyLife automatically observes supported activity, records what actually happened, dynamically reconciles the day, and preserves a trustworthy history.

V1 positioning: A local-first automatic personal planner and activity timeline. The user should not manually log ordinary phone/computer usage.

0. Scope Lock — What V1 Is and Is Not

V1 includes: Planner/Calendar, intentional activities/tasks, timer-required activities, automatic discovered activity, dynamic rescheduling, automatic timeline reconciliation, activity history, statistics, global search, day replay, contextual notifications, recurring planning, fixed/locked activities, time-leak detection, planned-vs-actual views, and robust local persistence.

V1 excludes: AI, Life Log, XP, levels, achievements, goals, notes, authentication, cloud sync, social features, theme system, and unnecessary gamification.

Do not add features outside this scope without explicit product approval. Reliability of the core tracking/planner engine is more important than feature count.

1. The Product in One Sentence

“Plan what matters, live normally, and let LogMyLife automatically reconstruct what actually happened.”

2. Fundamental Concepts

2.1 Planned Activity

Created intentionally by the user in Planner/Calendar. Examples: class, study, project, workout, meeting, travel.

2.2 Discovered Activity

Created by the system from supported device activity. Examples: Instagram 24m, YouTube 41m, Chrome 18m, VS Code 2h. The user does not need to create these.

2.3 Timer-Required Activity

A planned activity for which automatic app detection is not considered sufficient evidence. The user explicitly starts the timer. Example: DSA Practice. Opening VS Code must never automatically start DSA.

2.4 Session

A bounded period of activity with a source, start, end, duration, and evidence. Multiple sessions can belong to one activity.

2.5 Timeline

The chronological representation of planned and actual activity for a day. It is the primary user-facing story of the day.

3. Golden Rules

- Planner intent and automatic discovery are separate concepts.
- User must not manually log ordinary discovered app activity.
- Timer-required activities require explicit timer start.
- Never infer a timer-required task solely from an app being open.
- Never assign ambiguous activity to a planned task automatically.
- Unknown activity remains unknown until explicitly classified.
- One real-world minute must never be double-counted as two conflicting activities.
- Background execution is not automatically foreground usage.
- Notification delivery is not app usage.
- Locked/idle time does not count as active foreground work by default.

- Overtime is separate from planned duration.
- Rescheduling must preserve history and never silently destroy the original intent.
- Every derived record must be traceable to evidence or a user action.
- Statistics must be reproducible from stored source data.
- Never fake precision. If evidence is uncertain, show uncertainty.
- Core tracking must work locally without authentication or cloud sync.
- Do not build an always-running JavaScript polling loop for cross-app tracking.
- Android-native activity collection must be isolated behind a clean interface.
- UI should never be allowed to mutate raw evidence directly.
- Do not optimize for flattering productivity numbers; optimize for accuracy.

4. Canonical Planner / Calendar

Planner and Calendar are the same feature and must share one canonical schedule model.

A planner item should support title, start, end/duration, type, timerRequired, completion threshold where applicable, fixed/locked state, recurrence, metadata, status, and reschedule history.

```
planned_activity {
  id
  title
  start_at
  end_at
  duration_minutes
  type
  timer_required
  minimum_completion_percent
  fixed
  recurrence_rule
  status
  created_at
  updated_at
}
```

5. Planner Activity Types

Type	Created by	Automatic discovery	Timer
Normal planned	User	May be shown alongside it	Optional
Timer-required	User	Must NOT auto-start task	Required
Fixed event	User	Independent	Optional
Recurring	User	Each occurrence gets history	Per configuration
Discovered	System	Yes	No

6. Dynamic Scheduling Engine

6.1 Early Start

Planned 14:00–15:00, timer starts at 13:45. Default behavior: move the activity to 13:45–14:45 and cascade affected movable activities forward. The original schedule must remain in reschedule history.

6.2 Late Start

Planned 14:00–15:00, timer starts at 14:20. Move it to 14:20–15:20 and shift affected movable blocks.

6.3 Duration Preservation

Unless the user explicitly changes duration, starting early/late changes the start position but preserves the intended duration.

6.4 Fixed Events

Fixed/locked events cannot be shifted automatically. If a reschedule would collide with a fixed event, stop the cascade before the fixed event, find the next valid slot, or surface a conflict.

6.5 Cascading

A = movable activity
B = movable activity
C = fixed activity

If A moves and overlaps B:
move B to the next valid slot.

If B would overlap C:
do not move C.
place B after C if possible.
otherwise show unresolved conflict.

6.6 No Silent Overlap

The engine must never silently create two intentional planner blocks occupying the same time unless the user explicitly created an overlap and the product supports that mode.

7. Timer Behavior

Timer-required activity lifecycle:

```
SCHEDULED  
-> STARTED  
-> PAUSED (optional only if product supports pause)  
-> RESUMED  
-> STOPPED  
-> COMPLETED / INCOMPLETE
```

- Starting timer is an explicit user action.
- Timer session is authoritative for that activity.
- Timer may create a schedule shift if started outside the original window.
- Stopping early records actual duration; it does not fake completion.
- Running beyond the planned end creates overtime.
- Closing/restarting the app must recover the timer state from persistent timestamps.
- Do not depend on a JavaScript interval being alive for timer correctness; calculate elapsed time from timestamps.

8. 80% Completion Rule

For applicable timed activities, default threshold = 80%.

```
valid_duration / planned_duration >= 0.80 => Completed
```

Example:
planned = 120m
required = 96m
95m => Incomplete
96m => Completed
110m => Completed
150m => Completed + 30m overtime

The exact threshold must be stored per activity so future product settings do not require schema changes.

9. Automatic Activity Discovery — Android

Use Android UsageStatsManager / UsageEvents as the primary platform mechanism for supported app-usage observation. Usage access is controlled by Android and generally requires the user to grant Usage Access; the app must provide a clear permission onboarding flow. ■url■Android UsageStatsManager■<https://developer.android.com/reference/android/app/usage/UsageStatsManager>■

The native layer should collect usage events and periodically reconcile them. Do not rely on an always-running JS polling process. Android background and battery constraints must be respected.

The system must tolerate periods where usage data is delayed/unavailable, including reboot/unlock and permission changes. Missing data must be represented as unknown/unavailable, not fabricated.

9.1 Collector Contract

```
Native Activity Collector
-> normalized event stream
```

```
Event:
event_id
package_name
event_type
timestamp
source
device_state
received_at
```

9.2 Permission Loss

If Usage Access is revoked, stop claiming that automatic tracking is active. Show a clear status in the UI and continue supporting manual planner/timer functions.

10. Raw Events → Sessions

```
RAW EVENTS
|
+-- normalize timestamps
+-- remove duplicate events
+-- validate package/source
+-- debounce very short switches
|
SESSION BUILDER
|
+-- open session
+-- update session
+-- close session
|
IMMUTABLE SESSION RECORD
|
RECONCILIATION ENGINE
```

Sessions should not be stored as one continuously mutable counter. Store start/end evidence so the system can rebuild calculations.

11. Session Rules and Edge Cases

- App notification received but app not opened -> no usage session.
- App moves to background -> close foreground session.
- App opens for a few seconds -> apply debounce/minimum meaningful duration policy; do not create noisy statistics.
- Rapid A→B→A switching -> create bounded sessions, then optionally merge only where policy allows.
- Screen locked -> stop foreground activity counting.
- Device idle -> stop or mark session idle according to platform signal.
- Background audio/video -> do not count as foreground use by default.
- Phone call -> record as interruption/system activity, not automatically as productive or distraction.

- Unknown package -> preserve package identifier and mark classification unknown.
- Duplicate events -> deduplicate deterministically.
- Clock changes -> normalize using robust timestamps where available and detect suspicious jumps.
- Timezone change -> store timestamps in a canonical form and render in local time.
- Reboot -> recover what can be reconstructed; mark gaps rather than inventing activity.
- Battery saver/background restrictions -> show tracking health if collection becomes unreliable.

12. Discovered Activity vs Planned Activity

This is a critical distinction.

User plans:
14:00-15:00 Project

System observes:
14:10-14:35 Instagram

Result:
Project remains a planned activity.
Instagram becomes a discovered activity.
Instagram must NOT automatically be labeled as Project.

Automatic association is permitted only if an explicit, high-confidence rule exists. Otherwise discovered activity stays independent.

13. Activity Classification

High confidence

Explicit rule or directly bound timer session.

Medium confidence

Potentially related signal, but not sufficient for automatic productive credit.

Unknown

Insufficient evidence.

Only high-confidence evidence should automatically affect task completion. Medium-confidence data can appear in diagnostics/insights without inflating completion.

14. Interruptions

Interruptions should be first-class timeline records.

DSA Timer
10:00-10:27

WhatsApp
10:27-10:31

DSA Timer
10:31-11:00

Result:
DSA valid time = 56m
WhatsApp interruption = 4m

Do not label every interruption as a distraction. The system should distinguish interruption from judgment.

15. Actual Timeline

The timeline should combine intentional planner events, timer sessions, discovered activity, interruptions, idle gaps, and unknown gaps in chronological order.

```
09:00 Class ----- 09:54
09:54 Chrome ---- 06m
10:00 DSA Timer ----- 10:58
11:02 Instagram ---- 17m
11:20 YouTube ----- 23m
12:00 Lunch ----- 12:48
```

The user should be able to switch between **Planned** and **Actual** views.

16. Planned vs Actual View

Planned view answers: What did I intend to do?

Actual view answers: What actually happened?

The comparison view answers: Where did my plan diverge from reality?

Never overwrite the planned record just because actual behavior differed.

17. Activity Details / Historical Record

Clicking any activity must open its historical record.

Activity: DSA

Today

Planned: 60m

Actual: 58m

Completion: 96%

Sessions: 2

History

Aug 16 72m 92%

Aug 15 48m 80%

Aug 14 63m 100%

Lifetime

Total: 28h 42m

Sessions: 37

Average: 46m

Best session: 1h 48m

Historical records must survive planner edits. If an activity title changes, preserve historical identity using a stable activity ID.

18. Recurring Activities

Support recurring planner entries such as weekdays, specific days, or custom recurrence. Each occurrence must have its own occurrence identity and session history while retaining a link to the recurring template.

Editing one occurrence should not silently rewrite all past occurrences unless the user explicitly selects a future/all-occurrences operation.

19. Fixed / Locked Activities

Allow the user to mark activities as fixed. Examples: class, exam, appointment, travel.

- Fixed activities cannot be moved by automatic cascade.
- Movable activities may shift around fixed activities.
- Impossible schedules create a visible conflict instead of silent data corruption.
- Historical fixed times must remain intact.

20. Smart Schedule Recovery

When a timer activity starts early/late and displaces later movable blocks, cascade them to the next valid slots.

The engine should support a dry-run calculation before committing, so the UI can show the user the proposed shift.

```
Before:
14:00 DSA
15:00 Project
16:00 Workout
17:00 Fixed Class
```

DSA starts 14:20 and lasts 60m.

```
After:
14:20 DSA
15:20 Project
16:20 Workout
17:00 Fixed Class
```

If Workout cannot fit before 17:00:
move Workout to next valid slot or show conflict.

The scheduling algorithm must not endlessly cascade. Use a maximum iteration guard and deterministic slot search.

21. One-Tap Start Now

For a timer-required planner item, provide a clear Start Now action. It should invoke the same scheduling engine as a timer start; do not create a separate behavior path.

22. Time Leak Detection

Automatically identify unplanned time.

```
Total day
- intentional tracked time
- known planned/discovered classified time
= unallocated / unknown time
```

Show categories such as Instagram, YouTube, browsing, unknown, etc. Do not call time a 'waste' unless the user chooses that language; the system should report facts.

23. Day Replay

Provide a visual replay of the actual day. It should animate the chronological timeline using stored sessions, not generate fake transitions. The replay must be deterministic from history.

24. Best-Time Statistics

Without AI, calculate historical patterns such as longest sessions, average completion by hour/day, frequent interruption windows, and common unplanned activity periods.

Avoid causal claims. Say 'most successful sessions occurred between 9–11 AM', not 'morning makes you productive'.

25. Activity Categories

Discovered activities can be grouped into user-editable categories such as Social, Entertainment, Development, Education, Communication, Utilities, etc.

Category assignment is metadata; it must not automatically imply that an activity is good or bad.

26. Notifications — State Machine

UPCOMING
-> ACTIVE
-> INACTIVE / NOT_STARTED
-> NEAR_COMPLETION
-> COMPLETED
-> MISSED

Notify mainly on meaningful transitions.
Use cooldowns.
Never notify on every app switch.

- Upcoming activity reminder.
- Timer-required activity still not started.
- Near 80% completion.
- Completed.
- Schedule shifted.
- Conflict detected.
- Tracking permission lost.
- Tracking data temporarily unavailable.

Notification channels and runtime notification permission should follow the Android version's requirements. ■url■Android notification documentation■<https://developer.android.com/develop/ui/views/notifications>■

27. Global Search

Search should cover planned activities, discovered activities, historical sessions, and dates.

Search: DSA
-> Today 58m
-> Yesterday 72m
-> Aug 14 61m

Search: Instagram
-> all discovered sessions

Search: Aug 17
-> complete day timeline

28. Statistics Integrity

- All totals must be derivable from sessions.
- Do not maintain independent counters that can drift without a source.
- Use one canonical time unit internally, preferably milliseconds/seconds, and convert only at display.
- Handle timezone/day boundaries consistently.
- Do not double-count overlapping sessions.
- Separate planned, actual, unknown, interruption, and overtime buckets.
- Recalculate after edits using deterministic rules.

29. Local-First Persistence

V1 has no authentication and no cloud sync. The local database is the source of truth.

Persist raw activity events, derived sessions, planner activities, timer sessions, reschedule events, classification rules, and user corrections.

Use schema migrations. Never wipe user data during an app update.

30. Data Model

planned_activities

- id
- title
- start_at
- end_at
- duration_seconds
- type
- timer_required
- minimum_completion_percent
- fixed
- recurrence_id
- status
- created_at
- updated_at

activity_occurrences

- id
- planned_activity_id
- occurrence_date
- planned_start
- planned_end
- status

raw_activity_events

- id
- package_name
- event_type
- event_at
- device_state
- source
- created_at

activity_sessions

- id
- source_type (AUTO / TIMER / SYSTEM / MANUAL_CORRECTION)
- package_name
- planned_occurrence_id (nullable)
- started_at
- ended_at
- duration_seconds
- confidence
- classification
- created_at

reschedule_events

- id
- occurrence_id
- old_start
- old_end
- new_start
- new_end
- reason
- created_at

activity_rules

- id
- target_id
- rule_type
- rule_value
- confidence

user_corrections

- id
- session_id
- old_classification
- new_classification
- reason
- created_at

31. Reconciliation Algorithm

FOR each chronological time segment:

1. Normalize and validate source events.
2. Build non-overlapping sessions.
3. Determine source: AUTO / TIMER / SYSTEM / CORRECTION.
4. Locate planner occurrence covering the time.
5. If TIMER:
bind directly to its timer-required occurrence.
6. If AUTO:
record discovered activity.
7. If explicit high-confidence rule matches:
associate with planned activity.
8. Otherwise:
keep discovered/unknown.
9. Exclude idle/lock/background according to policy.
10. Calculate valid duration.
11. Calculate completion.
12. Calculate overtime.
13. Update timeline.
14. Emit notification only if a meaningful state transition occurred.

The algorithm must be deterministic and idempotent. Running reconciliation twice must not duplicate sessions or inflate time.

32. Idempotency & Recovery

If reconciliation runs after app restart, it must update existing records instead of creating duplicates. Every raw event/session needs stable identity or deterministic deduplication keys.

If the process crashes halfway through reconciliation, restarting must safely resume.

33. Clock / Time Safety

- Store canonical timestamps robustly; render local time in UI.
- Detect large backward/forward clock jumps.
- Never create negative session duration.
- Handle midnight-crossing sessions.
- Handle timezone changes without corrupting historical local dates.
- Use monotonic elapsed-time sources for active timers where appropriate, while persisting wall-clock timestamps for history.

34. Privacy & Permission UX

The activity tracker is inherently sensitive. The app must clearly explain why Usage Access is required, what is collected, and that V1 stores data locally.

Do not collect unrelated content such as message bodies, passwords, keystrokes, screenshots, or private text. Track the minimum metadata needed for the activity timeline.

If permission is unavailable, degrade gracefully instead of pretending tracking is active.

35. Performance / Battery Rules

- Do not poll aggressively.
- Batch/reconcile where possible.
- Do not keep a high-frequency JS timer alive just to measure elapsed time.

- Use timestamps for duration calculations.
- Process only the necessary time window during reconciliation.
- Archive/compact raw events only through a deliberate retention policy; never delete history unexpectedly.
- Prevent runaway session creation during rapid switching.

36. Security / Integrity Rules

- Validate all native events before persistence.
- Never trust package names as proof of task completion.
- Never expose raw tracking data unnecessarily in logs.
- Avoid logging sensitive app activity to development console in production.
- Keep database operations transactional where multiple records must change together.
- Reschedule + reschedule history + affected occurrences should commit atomically.

37. Edge-Case Master Checklist

- Timer started 30 minutes early.
- Timer started 1 minute early.
- Timer started late.
- Timer started then app killed.
- Timer survives device rotation/relaunch.
- Timer stopped early.
- Timer runs overtime.
- Timer crosses midnight.
- Two timers attempted simultaneously.
- Two planner blocks overlap.
- Movable activity collides with fixed activity.
- Multiple movable blocks must cascade.
- No free slot exists.
- Rapid app switching.
- App opened briefly.
- Notification received but app never opened.
- Background audio continues.
- Screen locked.
- Device idle.
- Phone call.
- System dialog.
- Unknown app.
- Permission revoked.
- Permission granted again.
- Device reboot.

- Usage data unavailable.
- Duplicate events.
- Clock changed.
- Timezone changed.
- Daylight-saving boundary.
- Activity edited after history exists.
- Activity deleted after history exists.
- Recurring template changed.
- One occurrence edited.
- App database migration.
- App crash during reconciliation.
- Battery saver delays processing.
- Very long session.
- Extremely large event history.
- User manually corrects a session.
- Reconciliation executed twice.
- App opened after several days.
- Unallocated time exists.
- Unknown time overlaps planned time.
- One minute could match multiple planned activities.

38. Testing Strategy

Unit Tests

Test scheduler, completion, session builder, deduplication, overlap detection, time calculations, recurrence and reconciliation with deterministic fake events.

Integration Tests

Simulate Android usage-event sequences and verify database + timeline output.

Property Tests

Useful invariants:

- Duration is never negative.
- Session totals never exceed their valid source intervals.
- Reconciliation is idempotent.
- Overlapping intentional activities never double-count.
- Completed cannot be true below threshold.
- Displayed completion never exceeds 100%.
- Reschedule never changes fixed events.
- Deleting/editing a planner item does not destroy immutable history unexpectedly.

Manual Device Tests

Test real Android devices across screen lock/unlock, reboot, permission changes, battery saver, background restrictions, app switching, notifications, and long-running days.

39. Antigravity Implementation Order

Phase 0 — Audit

Inspect the current LogMyLife code. Do not rewrite working screens. Map existing planner, calendar, timer, storage and navigation.

Phase 1 — Domain

Introduce canonical models and local persistence with migrations.

Phase 2 — Scheduler

Implement fixed/movable events, early/late starts, cascade shifting, conflict handling and reschedule history.

Phase 3 — Native Tracker

Implement Android UsageStats/UsageEvents collector, permission onboarding and tracking-health state.

Phase 4 — Sessions

Build event normalization, deduplication, session builder and idempotent persistence.

Phase 5 — Reconciliation

Separate planned/timer/discovered activity and implement confidence rules.

Phase 6 — Timer

Implement authoritative timer sessions and 80% completion.

Phase 7 — Timeline

Build Actual/Planned/Comparison timeline and historical details.

Phase 8 — Insights

Add statistics, time leaks, best-time patterns, categories and day replay.

Phase 9 — Notifications

Add state-machine notifications and cooldowns.

Phase 10 — Hardening

Run the complete edge-case suite, performance tests and recovery tests.

Phase 11 — UI Polish

Only after correctness: animations, micro-interactions, empty states and performance polish.

40. Acceptance Criteria — Final

- User can plan intentional activities without manually logging their actual duration.
- Normal app usage is automatically discovered without the user creating a planner item for it.
- Timer-required activities only track after explicit timer start.
- Early/late timer start reschedules the activity and shifts movable later planning.
- Fixed events are never silently shifted.
- Planner never silently creates conflicting intentional blocks.

- Clicking an activity shows current details and complete historical data.
- 80% threshold works exactly and is configurable per applicable activity.
- Overtime is separate.
- Unknown/ambiguous activity never inflates productivity.
- Notifications are contextual and not spammy.
- Day Replay reproduces the stored actual timeline.
- Statistics are deterministic and explainable.
- Reconciliation is idempotent.
- App restart does not lose active timer state.
- Permission loss is clearly surfaced.
- Reboot/data gaps are handled honestly.
- Local data survives app updates through migrations.
- No AI or excluded V1 feature is required for the core experience.
- Core user journey works without authentication or cloud sync.

41. UX Principles

- The user should do as little logging as possible.
- The planner should feel alive as the day progresses.
- Automatic discovery should be visible but never intrusive.
- Use clear labels: Planned, Actual, Discovered, Timed, Unknown, Interrupted, Overtime.
- Do not shame the user for discovered activity.
- Show facts before judgments.
- Every surprising statistic should have a drill-down explanation.
- Keep the main planner simple; advanced evidence lives in Activity Details.

42. What Antigravity Must NOT Do

- Do not rebuild the whole UI just to implement tracking.
- Do not add AI to solve deterministic problems.
- Do not assume an app equals a task.
- Do not use a simple foreground-service hack as the entire architecture without considering Android platform behavior.
- Do not store only aggregated totals.
- Do not silently delete or rewrite history.
- Do not make statistics from guessed data.
- Do not create duplicate sessions after reconciliation.
- Do not add authentication/cloud sync in V1.
- Do not implement every future feature before the tracking engine is reliable.

43. Definition of Done

LogMyLife V1 is done when a user can plan a day, live normally, manually start only the activities that require explicit timing, and later open the planner to see a trustworthy reconstruction of what was planned, what actually happened, where the schedule shifted, where time was unallocated, and the complete history of each activity — without manually logging ordinary activity.

44. Final Product Statement

LogMyLife is not a todo list with an activity tracker attached.

It is a dynamic personal timeline: the Planner represents intention, the Activity Engine represents observed reality, the Timer represents explicit intent when automatic inference is insufficient, and the Reconciliation Engine connects them without inventing facts.

Antigravity should treat this document as the V1 source of truth. If an implementation decision conflicts with a rule here, stop and surface the conflict rather than silently choosing a behavior.

45. Official Platform References

Android UsageStatsManager: [url](https://developer.android.com/reference/android/app/usage/UsageStatsManager) UsageStatsManager
reference <https://developer.android.com/reference/android/app/usage/UsageStatsManager>

Android notification documentation: [url](https://developer.android.com/develop/ui/views/notifications) Android
notifications <https://developer.android.com/develop/ui/views/notifications>